

Assignment 2

Issues & Fix:

1. Sliding-Window Overlap

HybridChunker splits at semantic boundaries, but a sentence at the end of a chunk and the beginning of the next chunk lose each other's context entirely. A retrieval query that spans that boundary returns an incomplete answer.

```
def add_overlap(
    texts: List[str],
    tokenizer,
    overlap_tokens: int = 128,
) -> List[Tuple[str, bool, bool]]:

    if overlap_tokens <= 0 or len(texts) < 2:
        return [(t, False, False) for t in texts]

    def last_n_tokens(text: str, n: int) -> str:
        ids = tokenizer.encode(text, add_special_tokens=False)
        return tokenizer.decode(ids[-n:], skip_special_tokens=True)

    def first_n_tokens(text: str, n: int) -> str:
        ids = tokenizer.encode(text, add_special_tokens=False)
        return tokenizer.decode(ids[:n], skip_special_tokens=True)

    result = []

    for i, text in enumerate(texts):
        prefix = last_n_tokens(texts[i - 1], overlap_tokens) if i > 0 else ""
        suffix = first_n_tokens(texts[i + 1], overlap_tokens) if i < len(texts) - 1 else ""

        parts = []
        if prefix:
            parts.append(f"[...prev] {prefix}")
        parts.append(text)
        if suffix:
            parts.append(f"{suffix} [...next]")

        result.append(("".join(parts), bool(prefix), bool(suffix)))

    return result
```

2. Metadata Missing

Each `.txt` chunk had minimal headers. Downstream ingestion code had no way to know which page a chunk came from, what type it was, or whether it overlaps with neighbours.

```
for i, chunk in enumerate(chunks, 1):
    base_name = f"{doc_stem}_chunk_{i:03d}"
    headings = list(chunk.meta.headings) if hasattr(chunk.meta, "headings") and
chunk.meta.headings else []
    page_numbers = extract_page_numbers(chunk)
    content_type = detect_content_type(chunk)
    token_count = len(self.tokenizer.encode(chunk.text, add_special_tokens=False))

    meta = ChunkMeta(
        chunk_id = hashlib.sha256(chunk.text.encode()).hexdigest(),
        source_file = original_name,
        doc_stem = doc_stem,
        chunk_index = i,
        total_chunks = len(chunks),
        token_count = token_count,
        headings = headings,
        page_numbers = page_numbers,
        content_type = content_type,
    )

# .txt
txt_path = output_dir / f"{base_name}.txt"
with open(txt_path, "w", encoding="utf-8") as f:
    if headings:
        f.write(f"Section: {' > '.join(headings)}\n")
    f.write(f"Source: {original_name}\n")
    f.write(f"Chunk: {i}/{len(chunks)} | tokens: {token_count}")
    if page_numbers:
        f.write(f" | pages: {page_numbers}")
    if content_type != "text":
        f.write(f" | type: {content_type}")
    f.write(f"\n{'-' * 60}\n")
    f.write(chunk.text)
    f.write("\n")

# .json
json_path = output_dir / f"{base_name}.json"
with open(json_path, "w", encoding="utf-8") as f:
    json.dump(asdict(meta), f, indent=2, ensure_ascii=False)
```

3. Strip Repeated Whitespace from Chunk Text

Docling sometimes produces chunks with multiple blank lines, trailing spaces, or `\n\n\n` gaps — this wastes tokens and confuses embeddings.

Fix: One-line cleanup before saving.

```
import re

def clean_text(text: str) -> str:
    text = re.sub(r'\n{3,}', '\n\n', text) # max 2 consecutive newlines
    text = re.sub(r'[ \t]+\n', '\n', text) # strip trailing spaces per line
    text = re.sub(r' {2,}', ' ', text) # collapse multiple spaces
    return text.strip()

# Use it before writing:
final_text = clean_text(chunk.text)
```

4. Skip Already-Chunked Files

If you re-run chunking after adding 2 new documents, all 50 existing files get re-processed — wasting time and overwriting good output.

Fix: Check if chunks for a file already exist before processing it.

```
def already_chunked(doc_stem: str, output_dir: str) -> bool:
    output_dir = Path(output_dir)
    existing = list(output_dir.glob(f'{doc_stem}_chunk_*.txt'))
    return len(existing) > 0

# In chunk_batch(), wrap the file loop:
for doc_path in files:
    if already_chunked(doc_path.stem, output_dir):
        logger.info(f"Skipping {doc_path.name} (already chunked)")
        continue
    chunk_count = self.chunk_file(doc_path, output_dir, file_mapping)
    results[doc_path.name] = chunk_count
```